Smart Contracts

Smart Contracts

- Contracts have evolved from being written on stone, wood, and paper to digital contracts in the computer age.
- Digital contracts are centralized, unreliable, and can be influenced or broken by powerful entities.
- Digital contracts are only legally enforceable, which can be manipulated by malicious parties.
- Blockchain enables decentralized, secure, and automatically enforceable contracts.
- No single party can alter or exert influence over blockchain-based contracts.
- This technology has the potential to positively transform the lives of billions of people.

Smart Contracts

Nick Szabo first theorized smart contracts in the 1990s, in an article called Formalizing and Securing Relationships on Public Networks.

- Smart contracts are described by Szabo as follows:

  A smart contract is an electronic transaction protocol that executes the terms of a contract. The general objectives are to satisfy common contractual conditions (such as payment terms, liens, confidentiality, and even enforcement), minimize exceptions both malicious and accidental, and minimize the need for trusted intermediaries. Related economic goals include lowering fraud loss, arbitrations and enforcement costs, and other transaction costs.

- The following is my attempt to provide a comprehensive generalized definition of a smart contract:

  A smart contract is a secure and unstoppable computer program representing an agreement that is automatically executable and enforceable.

Properties of Smart Contracts

A smart contract has the following properties:

- **Automatically executable:** It is self-executable on a blockchain when certain conditions satisfy coded instructions without requiring any intervention.
- **Enforceable:** This means that all contractual terms perform as specified and expected, even in the presence of adversaries. Preferably, smart contracts should not rely on any traditional methods of enforcement. Instead, they should work on the principle that code is the law, which means that there is no need for an arbitrator or a third party to enforce, control, or influence the execution of a smart contract.
- **Deterministic:** The deterministic feature ensures that smart contracts always produce the same output for a specific input. This property will allow a smart contract to be run by any node on a network and achieve the same result. If the result differs even slightly between nodes, then a consensus cannot be reached, and a whole paradigm of distributed consensuses on the block chain can fail.

Properties of Smart Contracts

- **Semantically sound:** This means that they are complete and meaningful to both people and computers.
- **Unstoppable:** This means that adversaries or unfavorable conditions cannot negatively affect the execution of a smart contract. When the smart contracts execute, they complete their performance deterministically in a finite amount of time.
- **Secure:** This means that smart contracts are tamper-proof (or tamper-resistant) and run with security guarantees. The underlying blockchain usually provides these security guarantees; however, the smart contract programming language and the smart contract code themselves must be correct, valid, and verified.

Oracles

- An inherent limitation of smart contracts is that they are unable to directly access any external data. The concept of oracles was introduced to address this issue.
- Oracles are an essential component of the smart contract and blockchain ecosystem. The limitation of smart contracts is that they cannot access external data because blockchains are closed systems, without any direct access to the real world.
- This external data might be required to control the execution of some business logic in the smart contract.
- Example: The stock price of a security product that is required by the contract to release dividend payments. In such situations, oracles can be used to provide external data to smart contracts.

Oracles

- An inherent limitation of smart contracts is that they are unable to directly access any external data. This external data might be required to control the execution of some business logic in the smart contract.

The concept of oracles was introduced to address this issue.

- Oracles are an essential component of the smart contract and blockchain ecosystem.
- An oracle can be defined as an interface that delivers data from an external source to smart contracts. Oracles are trusted entities that use a secure channel to transfer off-chain data to a smart contract.
- Example: The stock price of a security product that is required by the contract to release dividend payments. In such situations, oracles can be used to provide external data to smart contracts.

Oracles

The following diagram shows a generic model of an oracle and smart contract ecosystem:

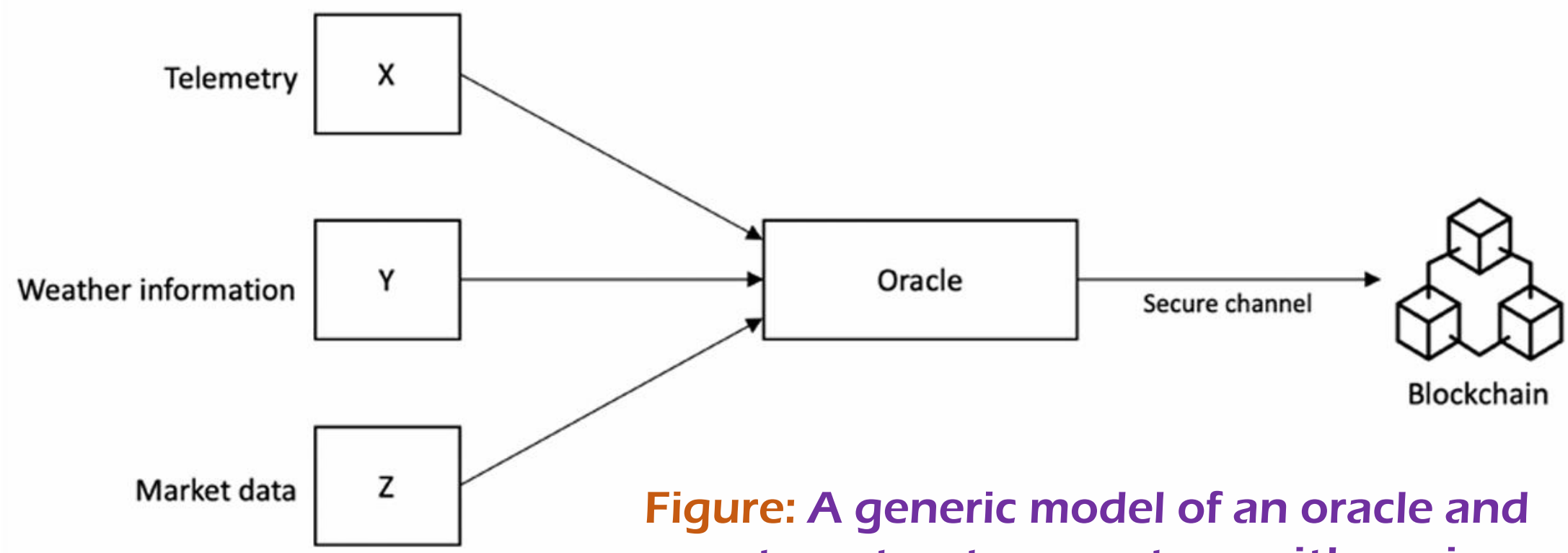


Figure: *A generic model of an oracle and smart contract ecosystem with various data sources*

Use-cases of Oracles

Depending on the industry and specific use-case requirements, oracles can provide various types of data, including weather reports, real-world news, corporate actions, and data from Internet of Things (IoT) devices.

Type of data	Examples	Use case
Market data	Live price feeds of financial instruments: exchange rates, performance, pricing, and historic data of commodities, indices, equities, bonds, and currencies	DApps related to financial services, for example, decentralized exchanges and decentralized finance (DeFi)
Political events	Election results	Prediction markets
Travel information	Flight schedules and delays	Insurance Dapps

Use-cases of Oracles

Type of data	Examples	Use case
Weather information	Flooding, temperature, and rain data	Insurance DApps
Sports	Results of football, cricket, and rugby matches	Prediction markets
Telemetry	Hardware IoT devices, sensor data, vehicle location, and vehicle tracker data	Insurance DApps Vehicle fleet management DApps

Access of Oracles into Blockchain

There are different methods used by oracles to write data into a blockchain, depending on the type of blockchain used.

- In a Bitcoin blockchain, an oracle can write data to a specific transaction, and a smart contract can monitor that transaction in the blockchain and read the data.
- Other methods include storing the fetched data in a smart contract's storage, which can then be accessed by other smart contracts on the blockchain via requests between smart contracts depending on the platform.
- In Ethereum, this can be achieved by using message calls.

Access of Oracles into Blockchain

The standard mechanics of how oracles work is presented here:

1. A smart contract sends a request for data to an oracle.
2. The request is executed, and the required data is requested from the source. There are various methods of requesting data from the source. These methods usually involve invoking APIs provided by the data provider, calling a web service, reading from a database, or requesting data from another blockchain. Sources can be any external off-chain data provider on the internet or in an internal enterprise network.
3. The data is sent to a notary to generate cryptographic proof (usually a digital signature) of the requested data to prove its validity (authenticity).
4. The data with the proof of validity is sent to the oracle.
5. The requested data with its proof of authenticity can be optionally saved on a decentralized storage system, such as Swarm or IPFS, and can be used by the smart contract/blockchain for verification.
6. Finally, the data, with the proof of validity, is sent to the smart contract.

Access of Oracles into Blockchain

- The diagram given below shows the generic data flow of a data request from a smart contract to the oracle.
- The oracle then requests the data from the data source, which is then sent to the attestation service for notarization.
- The data is sent to the oracle with proof of authenticity.
- Finally, the data is sent to the smart contract with cryptographic proof (authenticity proof) that the data is valid.

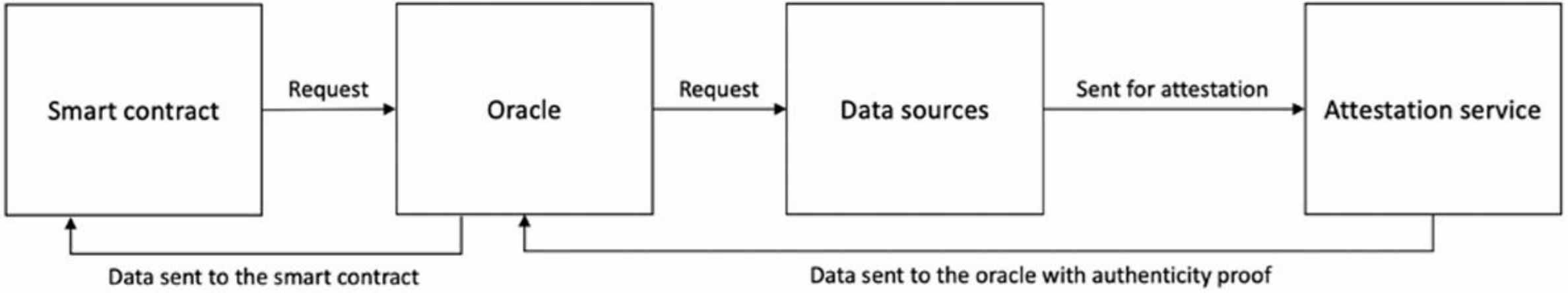


Figure: A generic oracle data flow